AESC v2: Two-Speed Memory Architecture (Revised Technical Report)

Oct 2, 2026 · @Giri

Author: Giridhari Karmakar · giriisdev@gmail.com

Abstract

AESC (Adaptive Event-State Computation) is a recurrent language model with a k-step neural field, a predictive dendrite, top-k skill routing, and two-speed memory. Fast weights are written token by token by a surprise-gated delta rule, and slow weights are trained by backpropagation and replay.

In a 9.92M-parameter reference implementation, the fast memory solved a 4-pair associative-recall task at 1.000 accuracy after 800 steps (chance 0.125). The first 10M implementation of this design did not: it scored 0.37 with fast updates on or off, and four of its modules received zero gradient.

This revision fixes eleven failure modes, corrects the formal propositions, and adds an observability layer for safety monitoring. It does not show that a 10M model matches a 100M model; Section 7 defines the test that would.

1. Introduction and hypothesis

The working hypothesis is that a small recurrent model with test-time memory can match a much larger transformer on memory-bound tasks at equal compute. This hypothesis is untested.

Transformers scale through attention stacks but have no built-in short-term plasticity. AESC instead carries a state and a fast-weight matrix across tokens, so the model can store and retrieve new associations while it reads.

The report states two testable claims:

1. **H1.** On associative recall, long-context copy, continual learning and in-context adaptation, a 10M AESC matches a 100M transformer at the same training data and the same FLOPs.
2. **H2.** On plain next-token loss, the 100M transformer wins; the goal is only to narrow the gap.

Contributions of this revision:

- A corrected design and equations (Section 3, Appendix A)
- A gradient policy for the fast write, backed by a measured ablation (Section 4)
- Results from a reference implementation and an audit of the first one (Section 7)
- A safety observability design (Section 6)
- An evaluation protocol for H1 (Section 7)

2. Related work

Each component has precedent; the contribution is the combination and the audit signals, not any single part.

- **Fast weights.** Schmidhuber (1992) and Ba et al. (2016) use a fast matrix that changes within a sequence. Schlag et al. (2021) show that linear attention is a fast-weight programmer, and DeltaNet (Yang et al., 2024) uses the delta rule that AESC′s write also uses.
- **Test-time memory.** Titans (Behrouz et al., 2024) writes memory driven by surprise, which is close to our gate.
- **State-space models.** Mamba (Gu and Dao, 2023) is the main recurrent baseline for the comparisons in Section 7.
- **Predictive coding.** Rao and Ballard (1999) motivate the dendrite error signal.
- **Two-speed memory.** McClelland, McNaughton and O′Reilly (1995) motivate fast learning plus replay into slow weights.
- **Looped computation.** Universal Transformers (Dehghani et al., 2018) reuse weights over steps, as the k-step field does.
- **Routing.** Sparse mixtures of experts (Shazeer et al., 2017) and straight-through estimators (Bengio et al., 2013) support the router.

3. Architecture

Each token passes through the blocks below in a fixed order, and only the fast memory is written by a local rule.

One recurrent state per token; only the fast memory is written locally

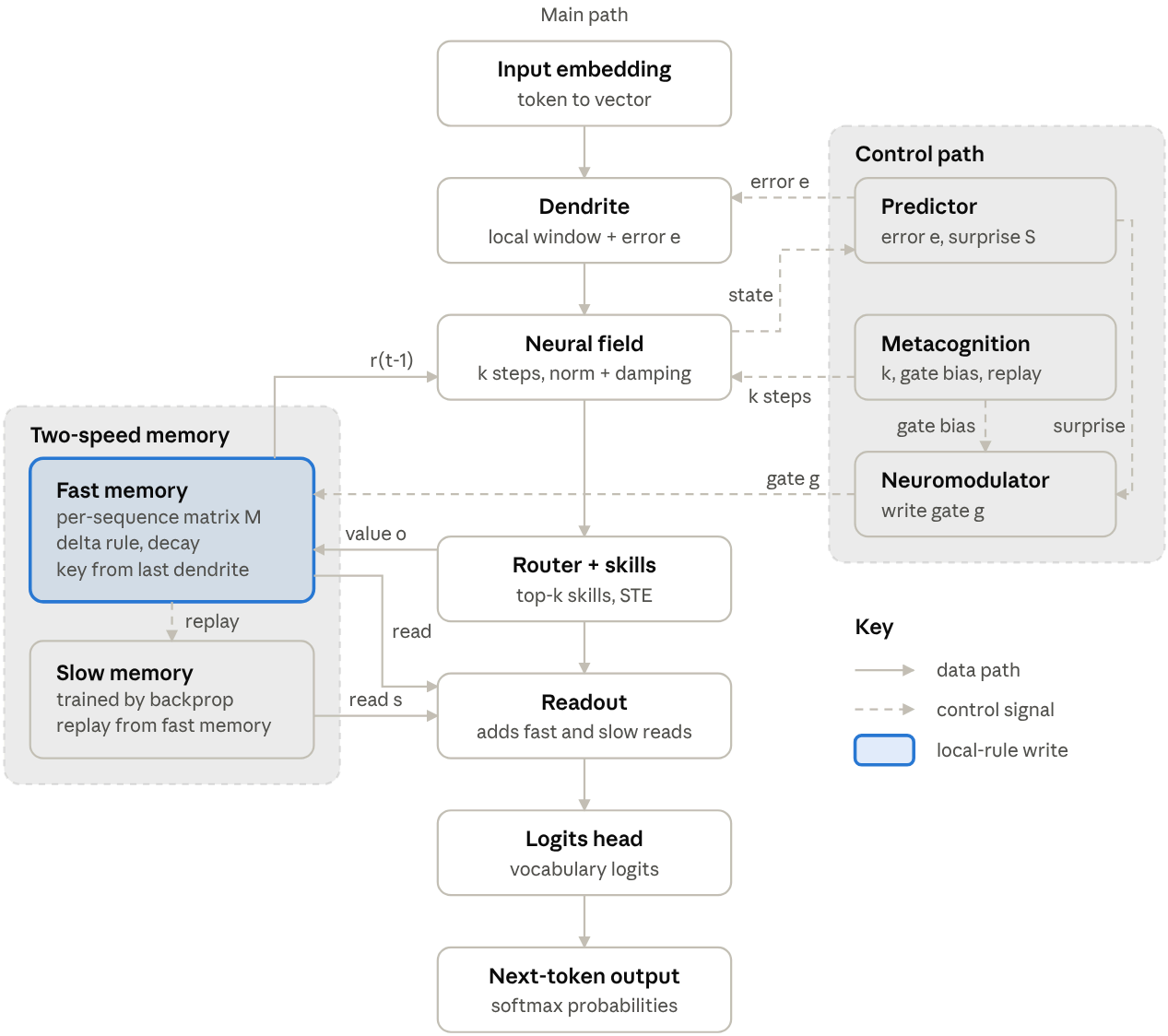


Figure 1 · AESC architecture · 9 blocks, main path, control path, two-speed memory

Tokens move down the center column; dashed arrows are control signals, and the highlighted fast memory is the only block written by a local rule.

Block	Role	Learning
Input embedding	Token to vector	Backprop
Dendrite	Local causal window; adds the previous step's prediction error	Backprop (next-token loss + error energy)

Block	Role	Learning
Neural field	k-step recurrent state with norm and damping; reads memory from the previous step	Backprop
Predictor, compare, surprise	Predicts the next input; surprise is the mean absolute error, detached	Backprop (error energy)
Neuromodulator	Write gate g from relative surprise and state activity	Gradient through the write (Section 4)
Router and skills	Top-k skills; learned weights kept in the forward pass	Backprop + straight-through
Fast memory	Per-sequence matrix, delta-rule write scaled by g, learned decay	Local rule in form; gradient to key, value and gate inside a truncated window
Slow memory	Consolidated weights read by a query	Backprop + replay
Metacognition	Chooses k, gate bias and replay timing from surprise statistics	Rules (a learned policy is future work)
Logits head	Vocabulary logits	Backprop

Timing rule (leakage-safe). The predictor forecasts x(t+1) from the state after token t. The error is computed when x(t+1) arrives and is used from step t+1 on. Nothing computed from x(t+1) may reach the logits that predict x(t+1).

Memory addressing. The write key comes from the previous step's dendrite feature, and the write value comes from the current skill output. The read query comes from the current dendrite feature, so a stored pair is retrieved when its key token appears again.

Figure 1 was redrawn. The v1 figure put the softmax before the logits head, had a garbled neuromodulator label, and showed an N-times stack with positional encoding. The new figure fixes the order and draws one recurrent state. Still to check with the author: that the block order above is confirmed.

4. Training procedure

Training takes one optimizer step per batch on the next-token loss, and the fast memory is written at every token. The write must stay inside the gradient graph for the key map, the value map and the gate.

Loss.

```
L = \sum_t \mathrm{CE}(z_t, y_t) + \lambda_{err}\,\lVert e_t \rVert^2 +
\lambda_{bal}\, B(\pi_t)
```

Here B is a router balance term that replaces v1's entropy term. The reference implementation uses lambda_err = 0.1 and lambda_bal = 0.01.

Gradient policy for the fast write. v1 (Proposition 3) put a stop-gradient on the write. That cuts all gradient to W_k, W_v and the gate. In the reference implementation, a stop-gradient write gave recall 0.75 against 1.000 with the write inside the graph, and the neuromodulator gradient was exactly zero. The policy is therefore to keep the delta-rule write differentiable inside a truncated window and to detach the state between windows.

Purely local credit assignment stays a research direction. If it is pursued, the neuromodulator needs its own auxiliary loss, for example a gate target based on how much the write lowers the next-token loss. That loss is a proposal and has not been tested.

Routing. The forward pass uses the softmax over the top-k logits, so unequal weights survive. The backward pass uses the gradient of the full softmax (straight-through). In a unit test, hard top-k gave router gradient to only the 2 picked skills out of 4, and straight-through gave it to all 4.

Replay and optimizer. Every 50 to 100 steps, Metacognition triggers replay: the slow weights move 2% toward the surprise-weighted average of the fast memories in the batch. The reference notebook uses AdamW with learning rate 1e-3, 100 warmup steps, cosine decay to 10%, and gradient clipping at 1.0. The 1e-4 to 3e-4 range in v1 was not tested. A curriculum of structured tasks first is still recommended and still untested.

5. Parameter efficiency

In the first 10M implementation, the embedding and logits head held 72% of the parameters, leaving about 2.8M for the architecture itself. A 10,000-token vocabulary at width 368 costs 3.68M in each of the two matrices.

Two fixes apply. Tie the input and output embeddings, which saves about 3.7M at that size, or use a small vocabulary (characters, or 2,000 to 4,000 subword tokens). v1 said subword tokens balance capacity; the opposite holds, because a larger vocabulary inflates both matrices.

Model	Total params	Largest block	Share
First implementation (vocab 10,000)	10.16M	Embedding + head	72%
Reference AESC (char vocab 65)	9.92M	Skills (8 operators)	85%

In the reference model the new blocks are small: the field has 2.6K parameters, the fast memory 0.33M and the slow memory 0.07M. Skills dominate, so a control with the same skills and none of the new blocks is needed before crediting the architecture for any gain.

Compute-matched protocol. Report parameters, FLOPs per token, training tokens and wall-clock time for every model. The k-step field multiplies the field's FLOPs by k, so compare at equal FLOPs as well as at equal parameters.

6. Safety and monitoring

AESC exposes internal signals that a monitor can log at every token. These signals are tripwires; they are not proof that the model's behavior is safe. This section is a design and has not been implemented or evaluated.

Signals logged per token.

- Surprise S(t) and its running average
- Write gate g(t), write norm, and the norm of the fast memory
- Router choices and router entropy
- Field steps k(t)
- Consolidation events into slow memory

Risks from test-time plasticity. The fast memory changes while the model is used. Content written in one session can steer later outputs (memory poisoning), the memory can drift without bound, and state shared across users can leak information.

Controls.

1. Fresh state for every session, with no carry-over between users.
2. Caps on the write gate, the write norm and the memory norm.

3. An audit log of writes: token index, gate, key norm and value norm.
4. An anomaly detector that flags signals far from a baseline distribution, for example a z-score on S(t), g(t) and router entropy.
5. A freeze switch that sets g = 0, so the memory only decays (Proposition 4), with an optional reset.
6. Replay into slow weights only from audited training sessions, never from live inference sessions, so a poisoned write cannot become permanent.

Evaluation plan. Inject adversarial text that tries to plant false associations. Measure the detection rate, the false-alarm rate, and whether a reset removes the effect.

Scope. A monitor on these signals can show that something changed, but not why, and it is not a faithful trace of the model's reasoning. Safety claims need red-team evaluation, which this report does not contain.

7. Experiments

The reference implementation solves associative recall with fast memory on, the first implementation did not, and no comparison against a 100M model exists yet.

All runs below are single-seed, small-configuration CPU runs. The recall task has 4 key-value pairs, 8 keys and 8 values, with the answer at the last position (chance 0.125).

#	Experiment	Model	Result
1	Recall, 800 steps	Reference AESC (d = 64)	Accuracy 1.000
2	Recall, fast memory removed, 300-step ablation	Reference AESC	Accuracy about 0.40
3	Recall, stop-gradient write, 800 steps	Reference AESC	Accuracy 0.75; gate and key-map gradient exactly 0
4	Recall, fast updates on or off, 500 steps	First implementation (d = 64, 4 skills)	Accuracy 0.37 to 0.39 either way

#	Experiment	Model	Result
5	Gradient audit, one backward pass	First implementation, 10.16M	Zero gradient in neuromodulator, metacognition, forgetting controller and field modulation input; router zero from the task loss
6	Gradient audit, one backward pass	Reference AESC, 9.92M	Nonzero gradient in all 11 blocks
7	Smoke run, char-level text, 200 steps, batch 8 x 32	AESC 9.92M, GRU 10.0M, Transformer about 10M	3.18, 2.91, 3.36 validation bits per character

Run 7 is too short to rank the models. Run 1 shows that the memory mechanism works. Removing the slow memory, neuromodulator, metacognition, router or predictor did not change recall in the ablation, because this task does not need them, so there is no evidence yet that those blocks help.

Protocol for H1.

1. Tasks: recall with distractors and 8 to 64 pairs, long-context copy, continual learning (train task A then B, measure the drop on A), and in-context adaptation.
2. Models: the 10M AESC, a 10M control with the same skills and none of the new blocks, a 10M transformer, a 10M Mamba or GRU, and a 100M transformer.
3. Same data, same training tokens, three or more seeds, mean and range reported, FLOPs per token listed.
4. H1 is supported only if the 10M AESC is within the seed range of the 100M transformer, or better, on these tasks. Otherwise H1 is rejected.

Open items. Harder tasks that exercise the slow memory, neuromodulator and metacognition; the 100M baseline; GPU timing (the 10M model ran at about 0.96 s per step at batch 8 x 32 on CPU, and GPU speed is unmeasured); and the safety evaluation in Section 6.

8. Failure modes and fixes

Eleven failure modes are known. Rows 1 to 8 come from the audit of the first implementation, and rows 9 to 11 from review of v1 and follow-up tests.

#	Failure mode	Evidence	Fix
1	Fast weights averaged over the batch	One matrix shared by all sequences (code audit)	Per-sequence memory of shape batch x d_k x d_v
2	Writes frozen at inference	Update guarded by the training flag	Gated writes allowed at inference, scoped to the session and audited
3	Neuromodulator off the graph	Zero gradient (audit)	Gate multiplies the differentiable write
4	Dendrite as autoencoder	It predicted features of its own input	Predict the next input; error arrives one step late
5	Router weights ignored	Selected skills averaged equally; zero router gradient from the task loss	Weighted skills with straight-through gradient
6	Field without memory read	Modulation input unused, zero gradient	Memory read fed into the field every step
7	Forgetting controller unused	Never called; zero gradient; fixed decay 0.95	Learnable per-key decay inside the write
8	Vocabulary swallows the budget	72% of 10.16M parameters	Tied embeddings or smaller vocabulary
9	Stop-gradient write blocks learning	Recall 0.75 vs 1.000; gate gradient 0	Differentiable write in a truncated window
10	Possible label leakage	v1 equation e(t) = x(t+1) - pred (design review, not measured)	One-step delay; unit test in Appendix B
11	Figure 1 errors	Softmax before logits head, garbled label, unexplained N-times stack	Redraw

9. Limitations and non-claims

No result here shows that a 10M model matches a 100M model on any task.

- The evidence comes from small configurations, single seeds and one synthetic task.

- The model processes tokens one at a time, so training is slower than a transformer of equal size; GPU throughput is unmeasured.
- Metacognition is a set of simple rules, not a learned policy.
- The safety layer is a design, not a tested system.
- We do not claim biological fidelity, convergence, unique skills or superiority to transformers.

Appendix A. Forward equations

Symbols: x_t is the input embedding, h the field state, M_t the fast memory, and sg the stop-gradient operator. The equations match the reference implementation.

Dendrite and prediction.

```
d_t = \tanh\Big(\sum_{j=0}^{w-1} c_j \odot x_{t-j} + b\Big) + \gamma \odot
e_t
```

```
\hat{x}_{t+1} = f_p(h_t), \qquad e_{t+1} = x_{t+1} - \hat{x}_{t+1}, \qquad
S_{t+1} = \mathrm{sg}\big(\mathrm{mean}\,|e_{t+1}|\big)
```

Field (repeated k_t times). R is a shared local convolution over neighboring neurons.

```
h \leftarrow (1-\zeta)\odot h + \zeta \odot \mathrm{LN}\Big(\alpha \odot h +
\beta \odot \big(d_t + R(\tanh h) + W_m r_{t-1}\big)\Big)
```

Control. The running average of surprise is \bar S, and the relative surprise is \tilde S_t = S_t / \bar S. The gate bias is delta_t = +0.5 if \tilde S_t > 1.5, -0.5 if \tilde S_t < 0.5, and 0 otherwise.

```
g_t = \sigma\big(a_s(\tilde S_t - 1) + a_c\,\overline{|h_t|} + b +
\delta_t\big)
```

```
k_t = 1 + \mathrm{round}\Big((k_{max}-1)\,\mathrm{clip}\big(\tfrac{\tilde S_t
- 0.5}{1.5},\,0,\,1\big)\Big)
```

Router and skills.

```
\pi = \mathrm{softmax}(W_r h_t), \quad w = \mathrm{softmax}(\text{top-}k\
\text{logits}), \quad w^{STE} = w + \pi - \mathrm{sg}(\pi), \quad o_t =
\sum_i w^{STE}_i\,\mathrm{Op}_i(h_t)
```

Fast memory. Lambda is a diagonal decay over key dimensions.

```
k_t = \frac{W_k d_{t-1}}{\lVert W_k d_{t-1} \rVert}, \qquad v_t = W_v o_t,
\qquad M_t = \Lambda M_{t-1} + g_t\, k_t \big(v_t - M_{t-1}^{\top}
k_t\big)^{\top}
```

Read and output.

```
q_t = \frac{W_q d_t}{\lVert W_q d_t \rVert}, \quad r_t = M_t^{\top} q_t,
\quad u_t = (W'_q d_t)^{\top} W_s, \quad z_t = W_o\,\mathrm{LN}\big(o_t + P_f
r_t + P_s u_t\big)
```

Replay into slow memory.

```
W_s \leftarrow (1-\eta)\,W_s + \eta \sum_b \frac{S^{(b)}}{\sum_{b'}
S^{(b')}}\, M_t^{(b)}
```

Appendix B. Design checklist

1. Per-sequence fast state of shape batch x d_k x d_v.
2. The gate multiplies the write; writes are allowed at inference and are audited.
3. Router weights are used in the forward pass, and every skill receives a straight-through gradient.
4. The dendrite predicts the next input, and the error is computed one step late.
5. The field reads the memory at every step.
6. The neuromodulator and gate receive gradient: after one backward pass on the next-token loss alone, their gradient norm is above zero.
7. The recall test passes (accuracy of at least 0.95 on 4-pair recall) before any other claim is made.
8. Leakage test: change x(t+1) and check that the logits at step t do not change.
9. The stop-gradient ablation is reported next to the differentiable write.
10. A per-block parameter report is included, with embedding and head at 25% of the total or less (a target), or tied.
11. Baselines are compared at equal FLOPs as well as equal parameters.
12. The safety log fields are present: S(t), g(t), router ids, k(t), the memory norm and the write audit.

Appendix C. Propositions

These follow from the chain rule and the construction. They are not convergence results.

Proposition 1 (reachability). If a module's output enters the logits or the error through a differentiable map with a nonzero Jacobian, the next-token loss gives that module a gradient that can be nonzero. Evidence: Run 6 shows nonzero gradient in all 11 blocks of the reference model.

Proposition 2 (straight-through routing). Let w be the softmax over the top-k logits and pi the full softmax. Then w + pi - sg(pi) equals w in the forward pass, and its gradient is the gradient of pi. Every skill therefore receives a router gradient. This replaces v1's uniform 1/k forward weights, which cannot express unequal skill weights.

Proposition 3 (revised: stop-gradient writes block learning). If the write M_t = Lambda M_{t-1} + g k (v - M^T k)^T is computed under stop-gradient, then M_t is constant with respect to every parameter, and W_k, W_v and the gate parameters receive no gradient through M. Observation: in Run 3 the gradient was exactly 0 and recall fell from 1.000 to 0.75. v1 claimed the stop-gradient design was justified; that claim is withdrawn.

Proposition 4 (zero gate). If g_t = 0 then M_t = Lambda M_{t-1}, so the memory only decays.

Proposition 5 (inference plasticity). With fixed parameters and g_t > 0, M changes across tokens during inference. This is why Section 6 requires per-session state and a write audit.

Proposition 6 (error supervision). The term lambda_err times the squared norm of e(t+1) sends gradient to the predictor and, through the state, to the field and dendrite. Surprise S is detached, so it carries no gradient.

Proposition 7 (no leakage). If the logits z_t depend on the inputs only through x(1..t), and errors are computed with a one-step delay, then the derivative of z_t with respect to x(t+1) is zero. The reference implementation processes one token per step, so causality holds by construction; the Appendix B test checks it.